University of East Anglia
COMPUTING SCIENCES

Advanced Web Development

Excise 3.0: Next.js Basics and Axios API

Welcome to Lab 3! In Lab 2, we built solid foundational knowledge using React to handle components, props, and UI states. While React is an excellent library for building interactive user interfaces, it leaves several architecture decisions up to the developer, such as routing, optimisation, and data fetching strategies.

Next.js is a production-ready framework built on top of React. If React provides the engine, Next.js provides the entire sports car. It abstracts away complex configurations and adds powerful, out-of-the-box features like:

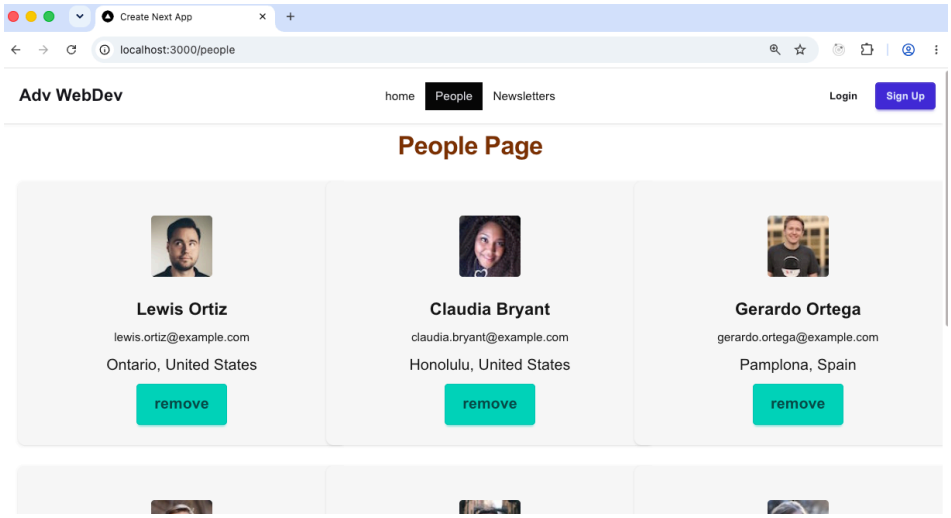
- File-system Routing: No need to install external libraries like react-router-dom. You create a folder, and it automatically becomes a URL route.
- Server-Side Rendering & Static Site Generation: React components traditionally render entirely in the user's browser (Client-Side Rendering). Next.js allows components to render on the server, sending pre-built HTML to the browser. This dramatically improves search engine optimisation and initial page load speeds.
- TypeScript Integration: Next.js has first-class support for TypeScript, offering built-in types, automatic configuration, and immediate feedback on data shapes.

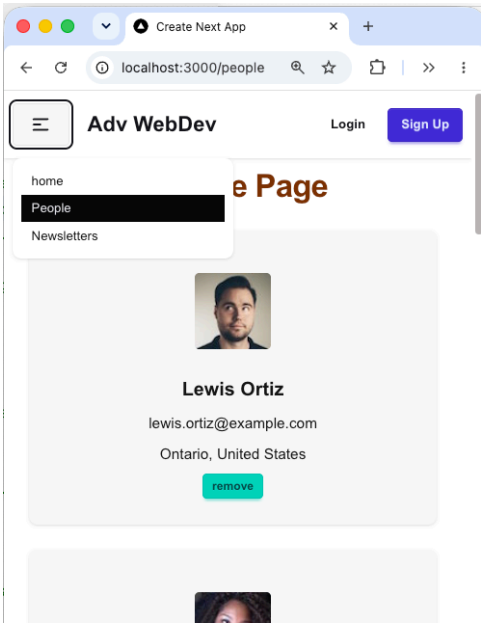
In the modern web ecosystem, companies rarely deploy vanilla React for customer-facing web apps. Learning Next.js teaches you how to design high-performance, full-stack applications with optimal rendering strategies, making it one of the most highly sought-after skill sets in frontend development today!

For this exercise, we will explore NexJS setup, understanding the file structure, and create routes and navigation. We will look at how to code a client-side component, including dynamic file-system routing and event handling, and daisyUI for styling. We will utilise React's `useState` and `useEffect` hooks for client components. While `useState` is essential for maintaining local state within a specific component tree, real-world applications often require a mechanism to share and synchronise state across many independent components. `useEffect` hook on the other hand is a built-in tool that allows functional components to perform "side effects", which are operations that interact with the outside world or occur outside the standard step-by-step UI rendering process. Common examples include fetching data from an API, manually updating the DOM, or setting up subscriptions and event listeners. Because these actions can disrupt the visual rendering of a component, `useEffect` hook deferentially executes them after the browser has finished painting the screen. By default, it runs after every single render, but it can be precisely controlled using a second argument called a "dependency array". Passing an empty array ([]) ensures the effect runs only once when the component first appears (mounts), which is crucial for actions like data fetching to prevent infinite loops and unnecessary server requests. We will use `useEffect` hook to fetch our data in the lab exercises.

Note: This lab heavily relies on concepts introduced in Lab 2 (React), including JSX, functional components, REACT hooks, and props. If you have not successfully completed and submitted Lab 2, please pause and complete it before attempting this lab.

Examples of final outputs:





Learning Objectives:

- Able to create a NextJS application using a build tool
- Able to implement file-system routing
- Able to code a client-side component in NextJS.
- Able to code API request using Axios API
- Able to implement responsive UI and apply daisyUI
- Practice React hooks.

Tasks:

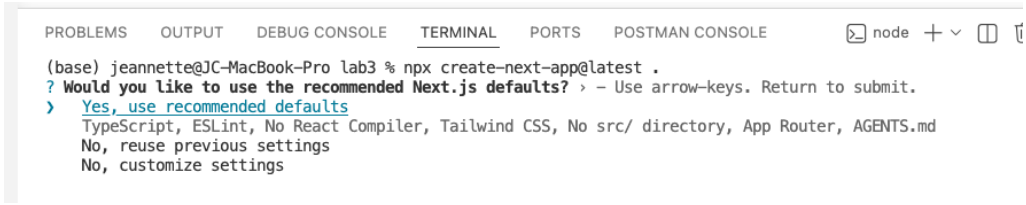
1. Create a new folder called `lab3`. Open this folder on VScode.



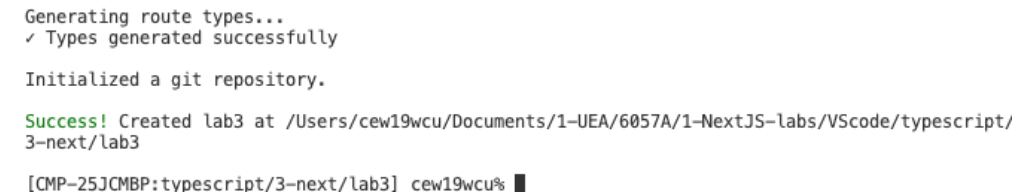
2. To create a Next project, use the following command, I called my app “lab3’, you can call yours anything you want.



3. Accept default configurations - - TypeScript, ESLint, No React Compiler, Tailwind CSS, No src/ directory, App Router, AGENTS.md



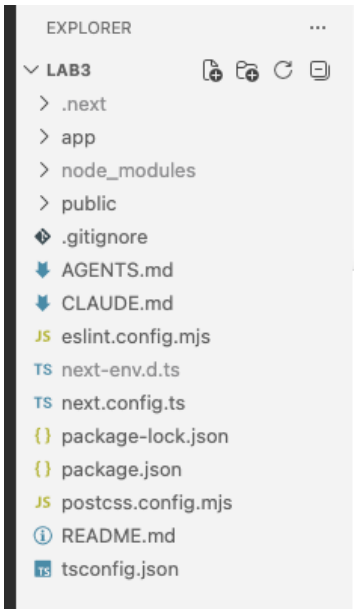
4. Once this is done, you should see this message:



5. We will use daisyUI, now install it as well:
   `npm i tailwindcss @tailwindcss/postcss daisyui@latest`



6. Examine the project's structure:



Next.js utilises a file-system-based routing architecture called the App Router. Every folder created inside the `app` folder automatically maps to a URL path in the browser. The `layout.tsx` file is the root layout, serves as the global HTML shell for your entire application. It defines the `<html>` and `<body>` tags, handles global fonts, and injects SEO metadata. Any UI defined here (like a navigation bar or footer) stays persistent across page changes. The `page.tsx` is the home page file. This is tied directly to the root path (/). When users visit http://localhost:3000, this component is what they see. The `globals.css` file houses your global CSS rules and the core utility directives for Tailwind CSS (@tailwind base;, etc.).

The public folder is reserved for hosting static files that do not change and do not need to pass through the Webpack/Turbopack compilation process. Files in this folder can be referenced directly from the root URL. For example, an image located at `public/logo.png` can be loaded in your code simply via `<Image src="/logo.png" />`. Common contents such as website favicons, SVG graphics, branding assets, and custom local font files are stored there.

The `tsconfig.json` file is used for TypeScript Configuration. This file dictates how TypeScript compiles your code into vanilla JavaScript. Because Next.js has native TypeScript support, it automatically manages this file for you.

The `next.config.ts` is used for Next.js Framework Configuration. This is where you customise advanced Next.js settings. Written in TypeScript, it alters how Next.js compiles and optimises your application. For example, we can set up remote domain permissions to optimise external images (e.g., loading profile pictures from an external API), enabling experimental framework features, or configuring environment redirect rules or set up the app environment variables.

The `package.json` file is the project manifest. The blueprint of your Next project environment. It lists everything required to build and execute your app, including scripts to run or build the app, project dependencies and their versions as well.

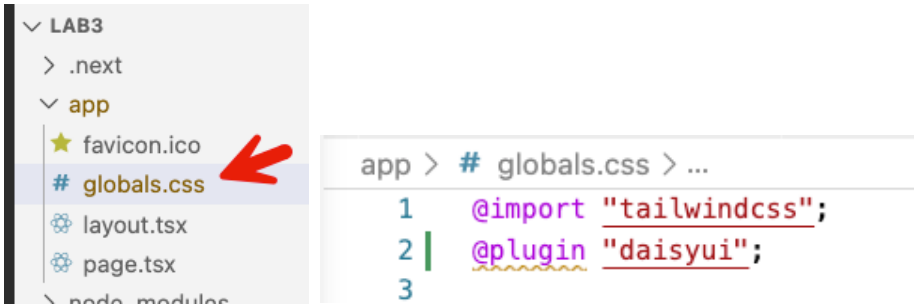
The `eslint.config.mjs` has information about the code quality linter. ESLint acts as an automated code reviewer. This configuration file ensures your entire team writes uniform, clean code. It uses the ECMAScript Module (.mjs) format to load code rules. Next.js extends its own custom rules here (eslint-config-next), helping catch structural React mistakes or broken Next.js rules (like using a standard HTML `<a>` tag instead of the optimised Next.js `<Link>` component) before you build.

The `AGENTS.md` is AI Agent Integration File. This file appears in modern project initialisations. It serves as a specialised markdown documentation file designed to brief AI coding agents and automated workflows.

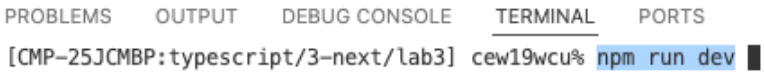
7. Open the `postcss.config.mjs` file, and check the Tailwindcss plugins:



8. We will add daisyUI plugin as well in `globals.css` file (line 2):



9. Bring up your development server:

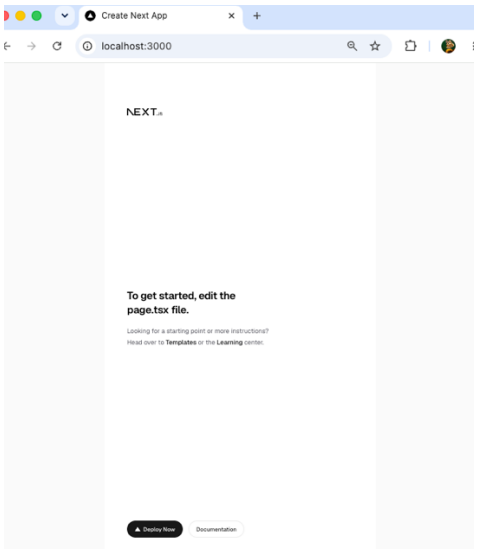


10. You will see that the server is running on port 3000:

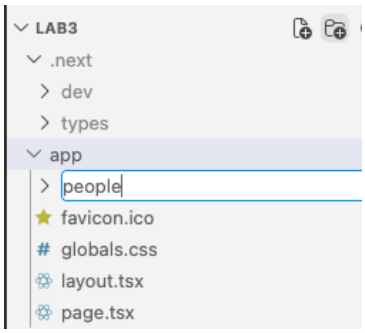


Notice that Turbopack is used by default to help speeding up the running of development server.

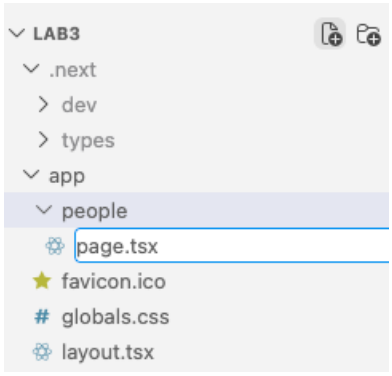
11. See the output on Chrome:



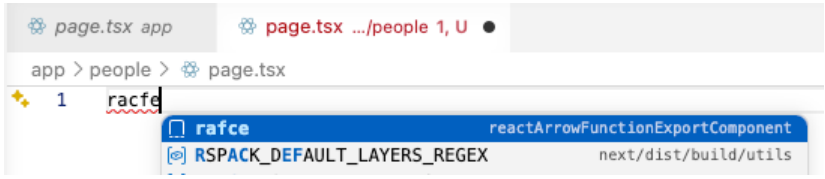
12. A simple file-system based routing is relatively straight forward to configure. Create a folder called “people” inside the ‘app’ folder:



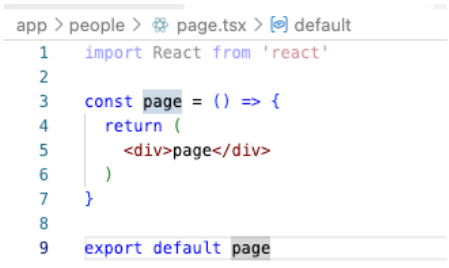
13. Inside this new folder create a new file called ‘`page.tsx`’



14. Open this “`people/page.tsx`” file and export the default function. If you have installed ES7+React snippets plugin tool, you can just simply type:



15. And the snippets will create a template code for you:



16. Now configure the heading for this page (line 5):

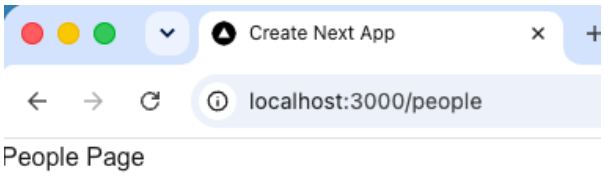
```
3   const page = () => {
4     return (
5       <div><h2> People Page</h2></div>
6     )
7   }
```

Notice that a functional component called “page” is created by default because the filename is `page.tsx`. You can change the name to anything you want, it will not affect the function of app. For me, I have changed mine to called “PeoplePage”, so that I can keep track on the component:

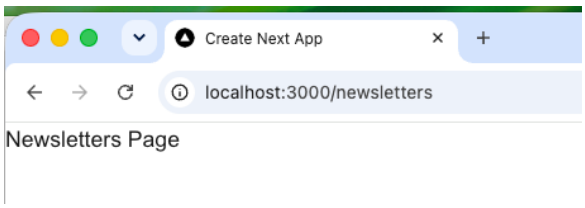
app > people > page.tsx > ...

```
1   import React from "react";
2
3   const PeoplePage = () => {
4     return (
5       <div>
6         <h2>People Page</h2>
7       </div>
8     );
9   };
10
11  export default PeoplePage;
12
```

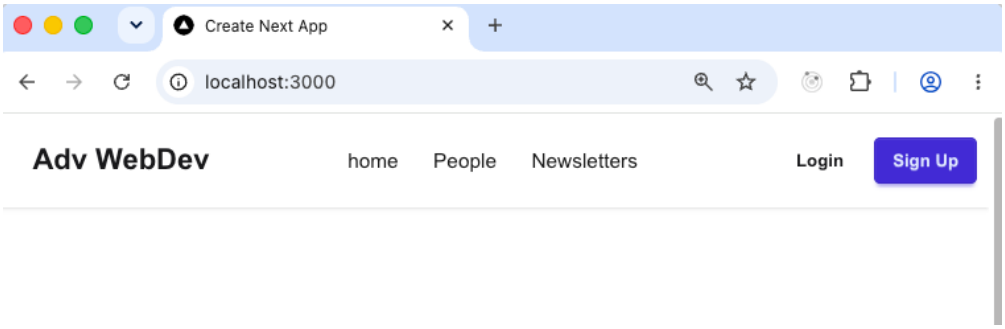
17. On Google Chrome navigate to http://localhost:3000/people , you will the text renders on the top left corner of the page.



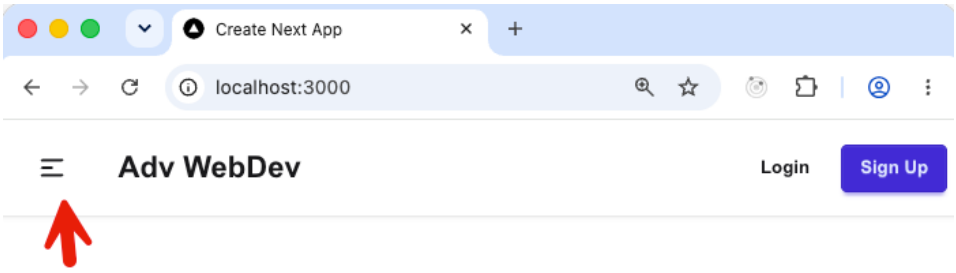
18. Now repeat the above steps (from task 10), but this time, the new folder is named “newsletters”, and test it on Google Chrome:



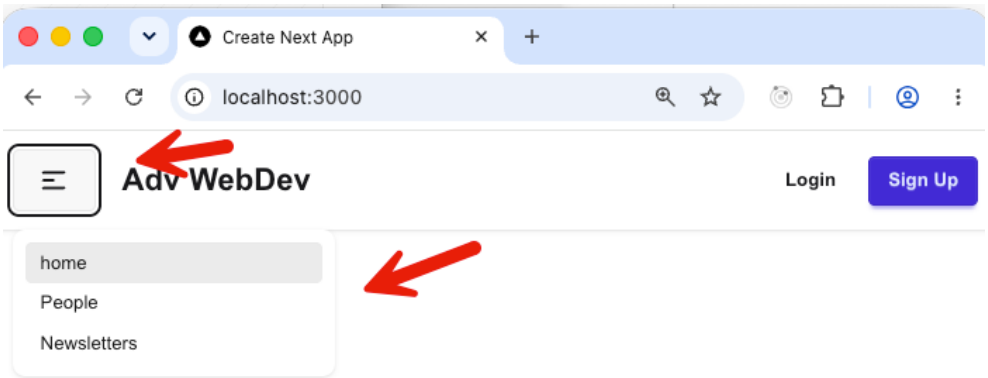
19. We have now created two routes - people and newsletter, in addition to the default home page. To navigate between these pages seamlessly, we need a navigation system. Here, we want to build our navigation system such that, on a big screen, there is a logo displayed at the top left-hand corner, links are displayed in the middle part of the navbar, and login and sign up links appear at the top right-hand corner, as shown below:



However, on a small screen, both the middle links will collapse and replace by a hamburger menu navigation system on the right side of the NarBar, as shown below:

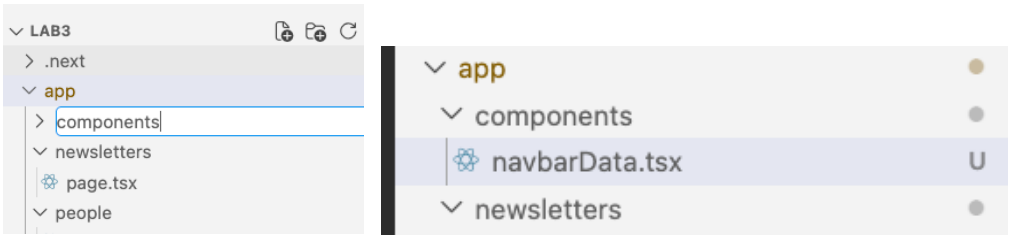


To response to user's interaction, for example, when click on the hamburger menu icon, the links will appear, as shown below:



We will implement the above behaviour next.

20. Before we build our `NavBar` component, we need to make an important architectural decision: Should this be a Client Component or a Server Component? In Next.js, any component that requires user interactivity (like toggling a mobile menu, tracking state, or responding to clicks) needs to be a Client Component. I like to separate out the data from code logic, so let's create the data we need for our `NavBar` first. In the `app` folder, create a new folder called "components" and in this folder, create a new file called '`navbarData.tsx`':



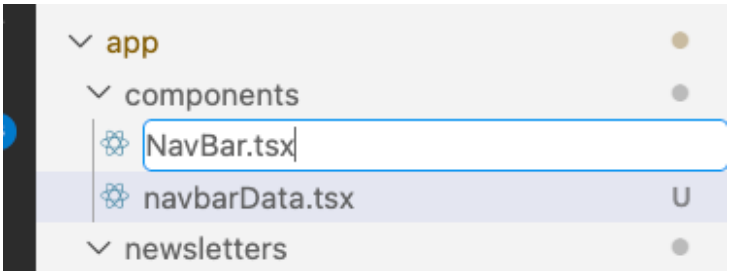
21. Open the '`navbarData.tsx`' file, first create the interface for our links. Each link has an `id`, `a url` and `text` as shown follow:

```
interface Link {
    id:number,
    url:string,
    text:string
}
```

22. We will create 3 links- Home, People and Newsletter pages respectively, as follow:

```
export const Links: Link[] = [
  {
    id: 1,
    url: "/",
    text: "home",
  },
  {
    id: 2,
    url: "/people",
    text: "People",
  },
  {
    id: 3,
    url: "/newsletters",
    text: "Newsletters",
  },
];
```

23. Next, create a new file called `NavBar.tsx`:



24. In this new file, export default this functional component (type `rafce` and select this option)

```
app > NavBar.tsx > default
1  import React from 'react'
2
3  const NavBar = () => {
4    return (
5      <div>NavBar</div>
6    )
7  }
8
9  export default NavBar
```

25. This `NavBar` is our client-side component, so declare this at the top of the file:

```
app > NavBar.tsx > NavBar
1   "use client"
```

26. Next, import the 'Links' from `navbarData`:

```
1  import {Links} from "./navbarData"
2
```

27. We also need the Link component from Next.js to help us create the links, so import it too:

```
2
3  import Link from "next/link";
4
```

28. Modify the `NavBar` component, and create the navigation links as follow:

```
const NavBar = () => {
  return (
    <nav>
        <ul>
          {Links.map((l) => {
            const { id, url, text } = l;
            return (
              <li key={id}>
                {/* here we use Link  */}
                <Link href={url}>{text}</Link>
                {/* <a href={url}>{text}</a> */}
              </li>
            );
          })}
        </ul>
    </nav>
  )
}
```

Question: What happens if we replace 'Link' with <a> ?

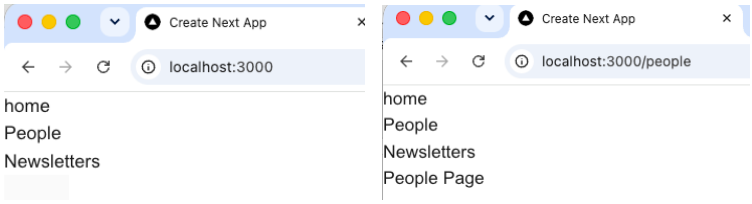
29. Now add this `NavBar` component to `layout.tsx`:

```
app > layout.tsx > geistMono
1   import type { Metadata } from "next";
2   import { Geist, Geist_Mono } from "next/font/google";
3   import "./globals.css";
4   import NavBar from "./components/NavBar";
5
```

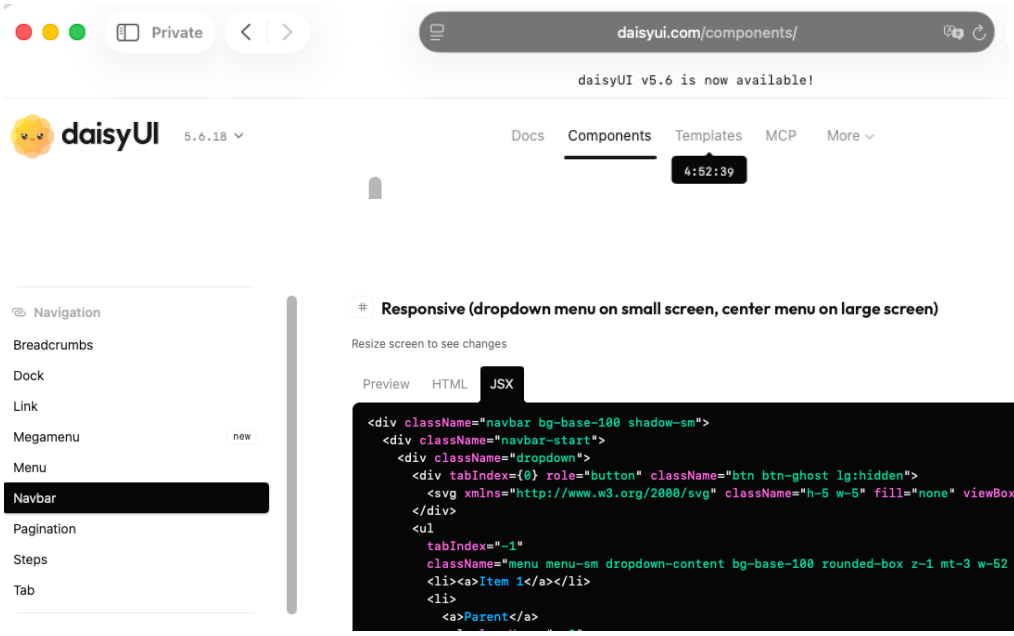
30. Scroll further down to the `return` statement, add the `NavBar` component but make sure to place it **above** the `children` node:

```
    <html
      lang="en"
      className={`${geistSans.variable} ${geistMono.variable} h-full antialiased`}
    >
      <body className="min-h-full flex flex-col">
        <NavBar />
        {children}
        </body>
    </html>
  );
```

31. Save the file, and see the results on Chrome:



32. Now, go to daisyUI page and select the responsive NavBar:



33. Modify the code to split the navbar into 3 sections: first is the logo (we called it Adv WebDev, then the links in the middle, follow by the end part which has 2 links – one for login one for sign up. Apply daisyUI navbar CSS to these sections. Below is my code:

    The main wrapper is `<nav>` tag

```
const NavBar = () => {
  return (
    <nav className="navbar bg-base-100 shadow-sm"> </nav>
```

The first section is styled in “narbar-start” (line 9 to line 44), inside the tag, there is a dropdown class for the hamburger icon, where, when click, a list of links will appear.

```
8   <nav className="navbar bg-base-100 shadow-sm">
9     <div className="navbar-start">
10      <div className="dropdown">
11        <div tabIndex={0} role="button" className="btn btn-ghost lg:hidden">
12          <svg
13            xmlns="http://www.w3.org/2000/svg"
14            className="h-5 w-5"
15            fill="none"
16            viewBox="0 0 24 24"
17            stroke="currentColor"
18          >
19            {" "}
20            <path
21              strokeLinecap="round"
22              strokeLinejoin="round"
23              strokeWidth="2"
24              d="M4 6h16M4 12h8m-8 6h16"
25            />{" "}
26          </svg>
27        </div>
28        <ul
29          tabIndex={-1}
30          className="menu menu-sm dropdown-content bg-base-100 rounded-box z-1 mt-3 w-52 p-2 shadow"
31        >
32          {Links.map((l) => {
33            const { id, url, text } = l;
34            return (
35              <li key={id}>
36                {/* here we use Link  */}
37                <Link href={url}>{text}</Link>
38              </li>
39            );
40          })}
41        </ul>
42      </div>
43      <a className="btn btn-ghost text-xl">Adv WebDev</a>
44    </div>
```

The middle section is styles as “narbar-center”, which renders the links. These links will disappear in large screens.

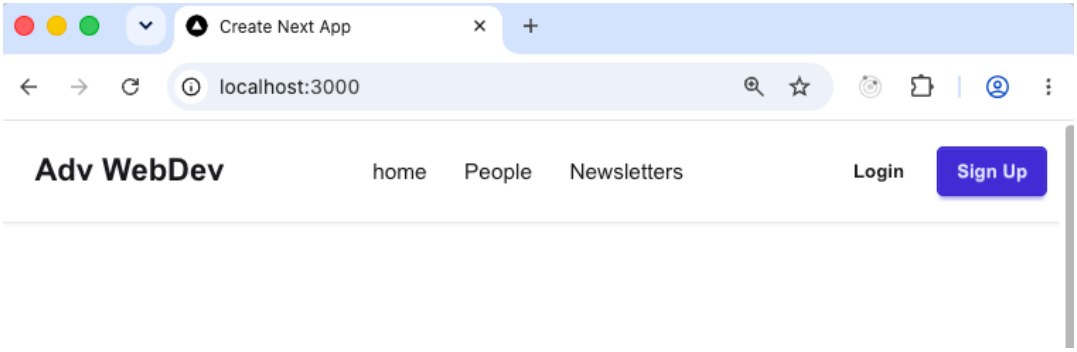
```
45      <div className="navbar-center hidden lg:flex">
46        <ul className="menu menu-horizontal px-1">
47          {Links.map((l) => {
48            const { id, url, text } = l;
49            return (
50              <li key={id}>
51                {/* here we use Link  */}
52                <Link href={url}>{text}</Link>
53              </li>
54            );
55          })}
56        </ul>
57      </div>
```

The last section is styled as “narbar-end”, where links to login and sign up appear.

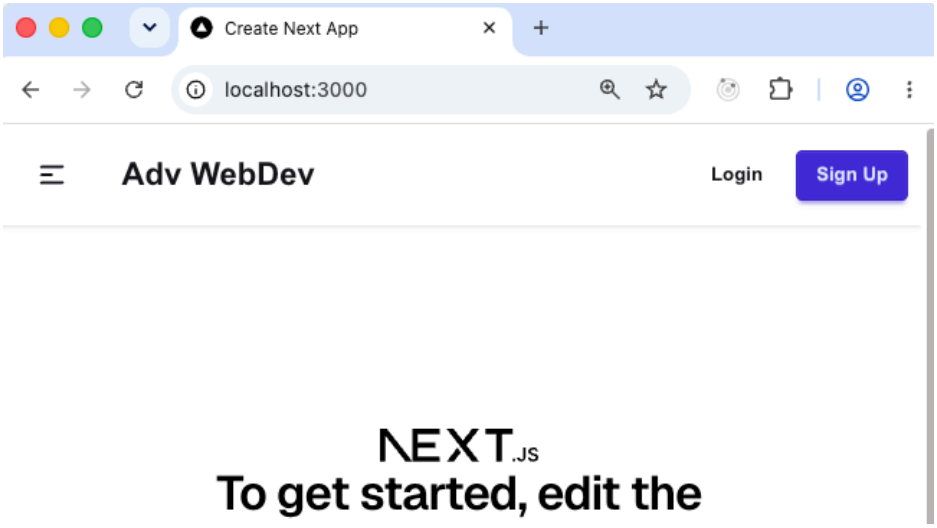
```
58      <div className="navbar-end">
59        {" "}
60        <div className="flex gap-2">
61          <Link href="/login" className="btn btn-ghost btn-sm">
62            Login
63          </Link>
64          <Link href="/register" className="btn btn-primary btn-sm">
65            Sign Up
66          </Link>
67        </div>
68      </div>
69    </nav>
```

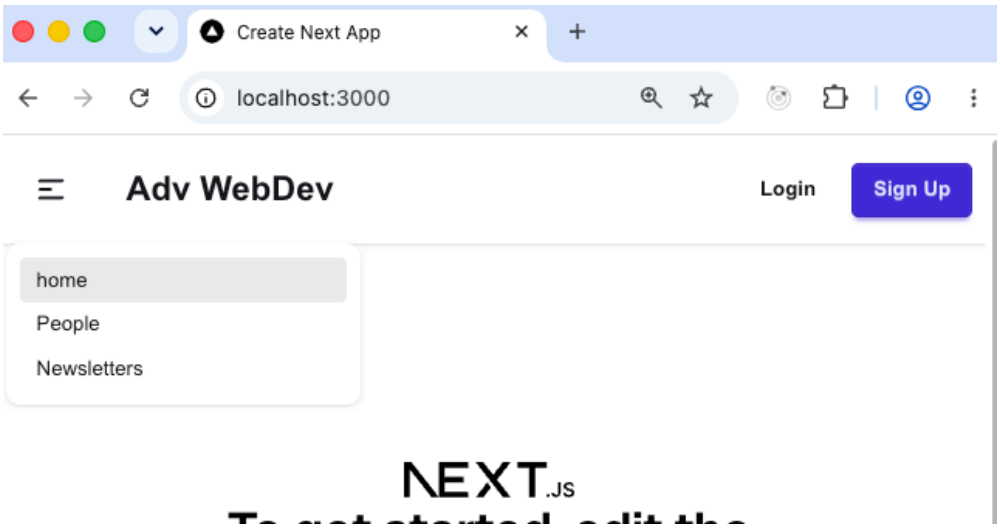
34. Check the output on Chrome:

Large screen output:



Small screen output:





Note: The login and sign up links do not work as we have not implemented them yet.

35. To configure active menu link, we will first import `usePathname` hook from `next/navigation`:

```
5   import { usePathname } from "next/navigation"; // for checking active links
6
```

36. And invoke the hook (line 8), make sure this is placed inside the component:

```
7   const NavBar = () => {
8     const pathname = usePathname();
9     return (
```

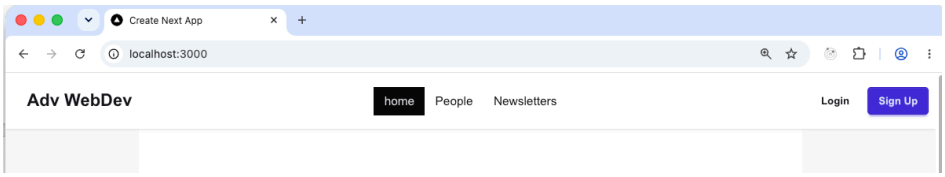
37. Apply this to the Link (line 56):

```
55              <li
56                className={`${pathname === url ? "menu-active" : ""}`}
57                key={id}
58              >
59                {/* here we use Link  */}
60                <Link href={url}>{text}</Link>
61              </li>
```

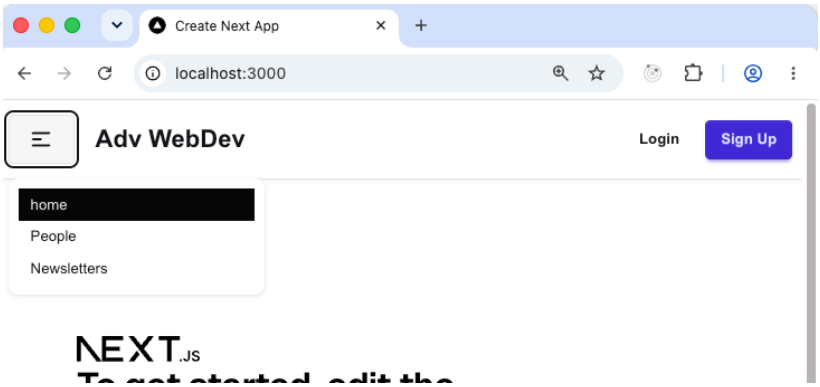
The code checks the current path to see if it is the `url` we coded for our nav links, if it is, set the class to “menu-active”.

38. Test it! below is my outputs for your information:

    Large screen:

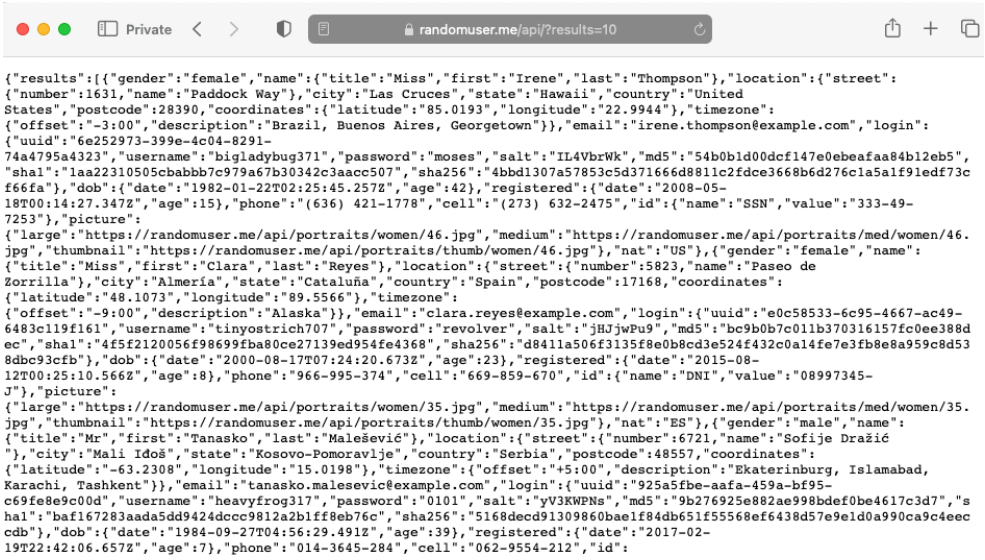


    Small screen:

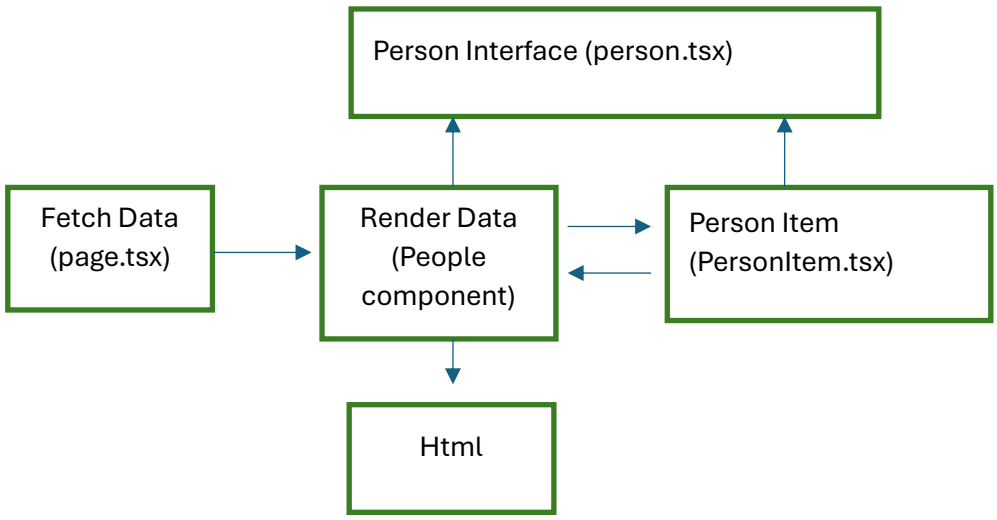


    We will implement client side fetching data next.

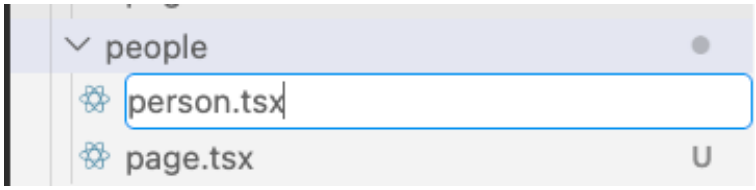
39. The following tasks involve the implementation of our People page. We will need `useState` hook , Axios API and `useEffect` hook. We will fetch the data generated by random user API : (https://randomuser.me/api/?results=10).



To handle the random user data, we will implement the following architecture:



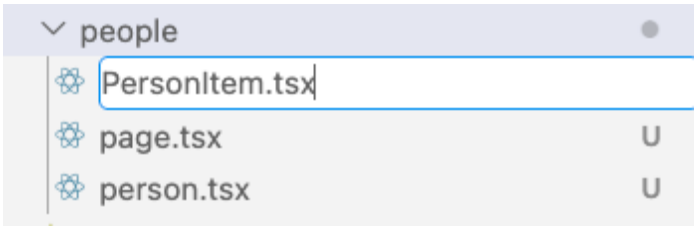
40. Inside the 'people' folder, create a new file called '`person.tsx`':



41. Similar to the tasks you have completed in lab2 - the book API React rendering, we will use the 'JSON to TS' plugin in VSCode to help us convert the random user JSON data to TS interface. So first copy the data shown on Chrome and save it as data.json and place this file inside the 'people' folder. Then follow lab2.0 task 34 to 40 to convert this JSON to TypeScript interfaces. Paste the TypeScript interfaces to this person.tsx. Examine the interfaces, you should have 12 interfaces (RootObject, Info, Result, Picture, Id, Dob, Login, Location, Timezone, Coordinates, Street, Name), and don't forget to export the type at the end of the file, as follow:

```
export type {Result, Name, Picture, Location, Login}
```

42. Next, we will implement the '`PersonItem`' component. Inside the 'people' folder, create a new file called '`PersonItem.tsx`', and export default functional component (hint: type rafce)



43. This file is a client component:



44. The component will render user's photo, name, email and location. We will import the 'Result' type:

```
1
2   import type { Result } from './person'
3
```

45. We will pass a remove function to this component, so type this function, as shown below. We will implement the function later.

```
4
5   interface PersonItemProps extends Result {
6     remove: (uuid: string) => void;
7   }
```

46. Modify the component as follow:

```
9   const PersonItem = ({ remove, ...result }: PersonItemProps) => {
10    const { email, login, location, name, picture } = result;
11
```

The code for the return body :

```
12    return (
13      <div className="" key={login.uuid}>
14        <img src={picture.medium} alt={name.last} />
15        <h2>
16          {name.first} {name.last}
17        </h2>
18        <h4>{email}</h4>
19        <p>
20          {location.city}, {location.country}
21        </p>
22        <button onClick={() => remove(login.uuid)}> remove </button>
23      </div>
24    );
```

47. Open the `page.tsx` file inside the 'people' folder. Fetching data from an external resource requires an asynchronous function. Here we will implement this as client side fetching using axios API. First, import 3 items – axios, Result interface and PersonItem component, as follow:

app > people > page.tsx > People

```
1   import axios from 'axios';
2   import type {Result} from './person'
3   import PersonItem from './PersonItem';
4
```

48. as we are using this as client component, declare this at the top of the file:

```
1   "use client"
2   import axios from 'axios';
3   import type {Result} from './person'
4   import PersonItem from './PersonItem';
5
6
```

49. The URL to random user API:

```
7
8   const url = "https://randomuser.me/api/?results=12"
9
```

Notice that I am fetching 12 records.

50. Here, we will only want to fetch the data once on the client side. To do this we will use React `useEffect` hook and we will use `useState` hook to manage the component states, including `isLoading`. So, import the hooks:

```
5
6   import { useEffect, useState } from 'react';
7
```

51. Inside the People component, initialise the states, one for the random users, one for isLoading:

```
27    const [users, setUsers] = useState<Result[]>([]);
28    const [isLoading, setIsLoading] = useState(true);
29
```

Notice that the loading status is initially set to true.

52. Then fetch the data using the `useEffect` hook, pass in a dependency array, as follow:

```
14
15    useEffect(() => {
16      axios
17        .get(url)
18        .then((response) => {
19          console.log(response);
20          setUsers(response.data.results);
21          setIsLoading(false);
22        })
23        .catch((error) => {
24          console.log(error.response);
25        });
26    }, []);
```

Add AbortController to clean up [Hint: see lecture notes]

53. To render the loading visual element, we simply check its current status:

```
28  if (isLoading) {
29    return (
30      <>
31        <span className="loading loading-bars loading-xs"></span>
32        <span className="loading loading-bars loading-sm"></span>
33        <span className="loading loading-bars loading-md"></span>
34        <span className="loading loading-bars loading-lg"></span>
35        <span className="loading loading-bars loading-xl"></span>
36      </>
37    );
38  }
```

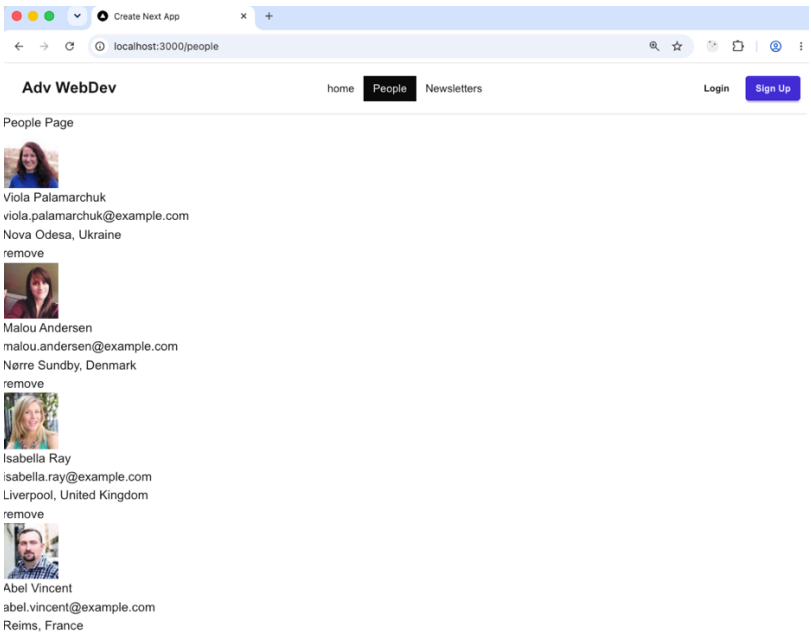
I will leave the CSS styling and loading UI to you to create. You can implement daisyUI loading classes: https://daisyui.com/components/loading/

54. We are now ready to code the remaining JSX, as follow:

```
40  return (
41    <section>
42      <h2>People Page</h2>
43      <div>
44        {users?.map((user) => {
45          return (
46            <PersonItem {...user} key={user.login.uuid} remove={() => {}} />
47          );
48        })}
49      </div>
50    </section>
51  );
```

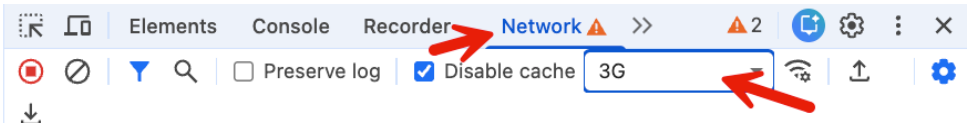
Notice that the state is called 'users' – refer to task 51 above.

55. Now test the page!



There is no CSS yet, but we will fix this later

56. Set Network to 3G to test the loader:



Also clear the cache :

Stop the server (^C), then type `rm -r .next`

```
 GET /people 200 in 41ms (next.js: 1792µs, application-
^C
(base) jeannette@JC-MacBook-Pro lab3 % rm -r .next
```

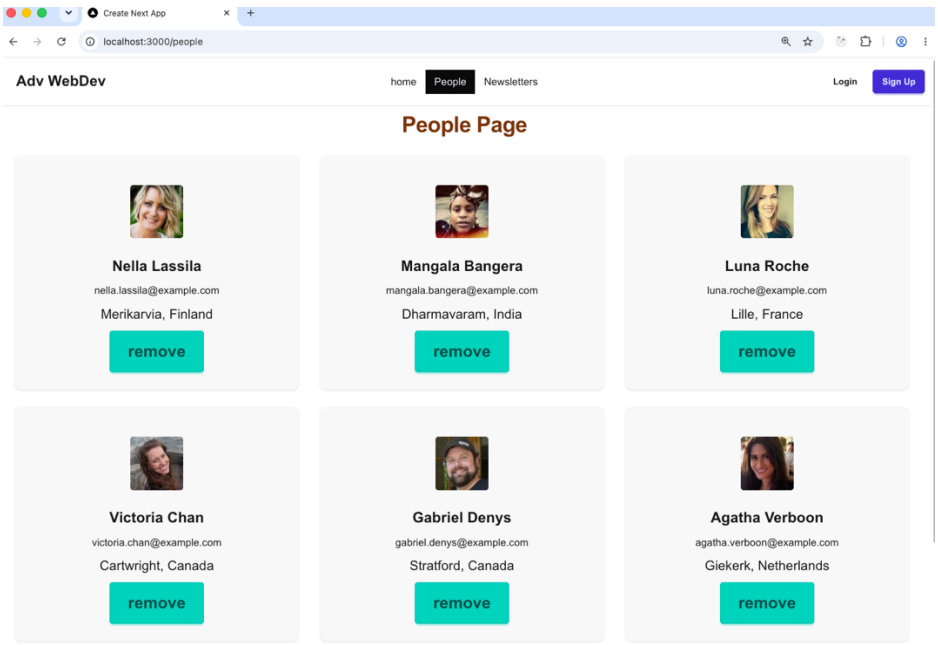
Restart the server

```
(base) jeannette@JC-MacBook-Pro lab3 % rm -r .next
(base) jeannette@JC-MacBook-Pro lab3 % npm run dev
```

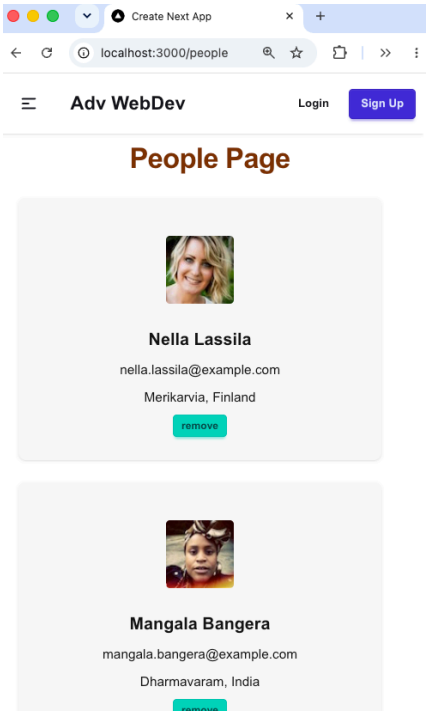
Test to see if you can see the loader!

57. Go to daisyUI site and select the “Card with centered content and paddings” https://daisyui.com/components/card/?lang=en , and apply it to users. My outputs below for your information:

Large screen:



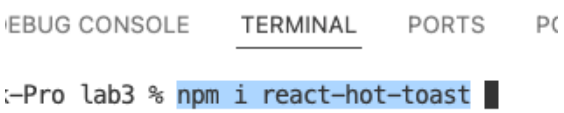
Small screen:



58. Once you are happy with your UI, we will test the incorrect URL. Change the URL of the random user API to something invalid (e.g. https://randomuser.me/apiii/?results=12) and see the errors on the console:



59. Here we can present this message to our user in a "friendly' manner by using React Toastify. So, stop the development server now, and install React Hot Toast:
    a. Stop the server – (^C)
    b. Install React Hot Toast https://react-hot-toast.com – (`npm install react-hot-toast`)



    c. Restart the server – (`npm run dev`)

60. Open your `layout.tsx` file, and add the Toaster inside the body tag, as follow:

```
return (
  <html
    lang="en"
    className={`${geistSans.variable} ${geistMono.variable} h-fu
  >
    <body className="min-h-full flex flex-col">
      <Toaster position="bottom-right" />
      <NavBar />
      {children}

    </body>
  </html>
);
```

Notice that I want my message to appear at the bottom right of the page, you can set a different location if you would like.

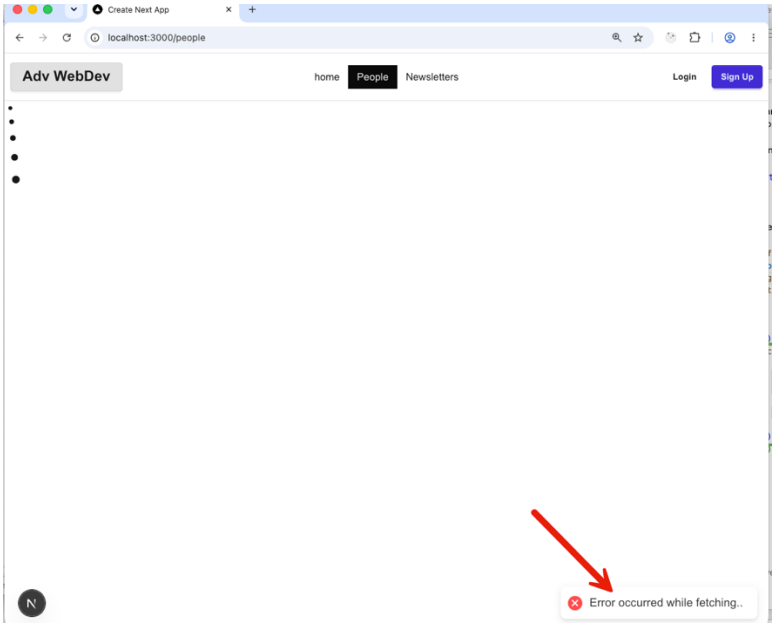
61. Obviously, you need to import the Toaster:

```
import { Toaster } from "react-hot-toast";
```

62. Then modify the catch block as follow (line 24 to line 31)

```
useEffect(() => {
  axios
    .get(url)
    .then((response) => {
      console.log(response);
      setUsers(response.data.results);
      setIsLoading(false);
    })
    .catch((error: any) => {
      if (error.isAxiosError) {
        console.log(error.response);
        toast.error(`Error occurred while fetching..`);
      } else {
        console.log(error);
      }
    });
}, []);
```

63. Once you have completed the above configurations, test it! Below is my output for your reference:



64. Correct the URL and test again.

Once you have achieved this point well done!

Your next task is to commit the code to CMP Stugit and name your repo as “<your-username>lab3.0”